

A Multithreaded DSM System with Thread Migration for Memory Locality over InfiniBand

by

Tse-ming Kao

THESIS

Presented to the Faculty of the Graduate School of
National Tsing Hua University
in Partial Fulfillment
for the Degree of

MASTER OF SCIENCE

Computer Science Department
TSING HUA UNIVERSITY

July 2005

Abstract

In a multithreaded system, whether the arrangement of resources properly or not can make a big difference on efficiency. It is even more remarkable for a distributed one, because the resources are more dynamic. A software DSM system is such an environment that needs good management to achieve high performance. This thesis presents a software DSM system that concerns about the relationship between memory pages and nodes. In this system, we try to group pages according to the access pattern of threads and aggregate the pages on the same node to reduce the times of page fault happened. However, for parallelism consideration, we could not group all pages on the same node to enhance the locality. Hence we add thread migration mechanism into the system while the pages are still distributed among nodes. We also adopted InfiniBand as the communication network. By using the features of InfiniBand, such as remote direct memory access (RDMA), scatter-gather functionality, etc., the performance can be improved further.

Table of Contents

Abstract	i
Chapter 1 Introduction	1
Chapter 2 Preliminaries	8
2.1 Overview of InfiniBand	8
2.2 Software DSM Systems	9
Chapter 3 Architecture of Our Systems	13
3.1 Threading Subsystem	13
3.2 Memory Management Subsystem	14
3.3 Communication Subsystem	16
Chapter 4 Memory Locality Concerned Approach	18
Chapter 5 Performance Evaluation	24
5.1 Experimental Environment	24
5.2 Performance Results	26
Chapter 6 Conclusions and Future Work	31

List of Figures

2.1	The problem of thread migration within RC model	12
4.1	The advantage of thread migration	19
4.2	The advantage of page fetching	20
5.1	The results of FFT	26
5.2	The results of SOR	28
5.3	The results of LU	28
5.4	The results of QS	29
5.5	The results of OCEAN	29

List of Tables

5.1	The costs of basic operations	24
5.2	Characteristics of the benchmark programs	25

Chapter 1

Introduction

A distributed shared memory (DSM) system is a loosely-coupled architecture that provides a single address space sharing among all nodes in a cluster. The DSM systems can be implemented in hardware or software. In general, the hardware one can provide better performance, but it is more flexible and convenient in software. With the modern techniques of advance network and IC design, the network of workstation is getting more popular and powerful. The software DSM seems to be a practical and relatively lost-cost platform for parallel computing. Programmers write programs in a DSM system can still have simple memory view as usual without being confused with the distributed environment. For simplicity of programming style, the shared memory is usually divided into pages. The local memory on different nodes can be treated as the cache. The copies of a page can be located on many nodes at the same time. Keeping the pages consistent is the most important job that a DSM system takes charge of.

The consistency model is an essential component of a DSM system that heavily affects the performance. The impact of performance in DSM systems is mainly introduced by false sharing and communication latency. To achieve high

performance, many different consistency models had been proposed. The release consistency (RC) [3] could be a proper type achieving efficiency by reducing the effects of false sharing. There are many researches that are based on the RC model with some modifications to enhance the efficiency further. The Home-based lazy release consistency (HLRC) [4, 19] is a great one which provides good results.

The major feature of HLRC is that each page is assigned with a home node, and all changes of the page are collected by its home node. It can simplify the management of page modification comparing to the homeless one. For HLRC, the performance can be decided by assignments of the home node. If the system doesn't have the capacity to concern about the assignments of the home of a page according to the access pattern of an application, it may result in poor performance. Even if the system got the capacity to re-assign the home, the decision may not be correct to improve performance. Dynamic adapting to the behavior of applications leads better results [8, 18].

In this thesis, we propose a scheme that uses HLRC protocol and can take advantages of memory locality. The basic thought is to make the data that a thread might be interested in being aggregated on some nodes. In this situation, a thread on one node might need to move itself onto the other node where the data is located. But we can reduce the computation due to page fault, and the communication to

transfer data by the method we proposed. Considering the overhead introduced by communication, a few kilo-byte stack and tens kilo-byte pages, if the assignment of pages is proper for access pattern, moving the thread can be cheaper.

The rest of this thesis is arranged as follows. Chapter 2 introduces the InfiniBand and the software DSM systems. Some brief introductions of the implemented DSM systems, MLDSM, will be given in Chapter 3. We also proposed a scheme that is used in our DSM system and will be described in Chapter 4. The performance results are presented in Chapter 5. The final chapter, Chapter 6, gives summarized conclusions.

Chapter 2

Preliminaries

In this chapter, a brief overview of InfiniBand and introductions of basic DSM components will be given in the following sections.

2.1 An overview of InfiniBand

InfiniBand is a channel based, switched fabric interconnect standard, and its most functionalities are implemented directly in hardware. It can efficiently relieve the stresses of CPU caused by lots of I/O operations. The implements of InfiniBand are prominent in server clusters where high bandwidth and low latency are key requirements. The InfiniBand protocol throws away the traditional "load store" model, and adopts message-passing structure for more efficient and reliable transfer of data.

The software transport verbs defined in InfiniBand architecture specifications describe the interface between the host channel adapter (HCA) and the operating system. For InfiniBand, besides the traditional communication operations, like send and receive, it also offers some other advanced operations. InfiniBand architecture (IBA) supports memory semantics. The hardware-based remote memory access operations are used to replace the traditional send and receive operations. These remote operations are called remote directly memory access (RDMA), including

remote read/write, atomic compare and swap, and atomic fetch and add. These operations can access remote memory space over the network without the interfering of the receiver. This feature is important for scalability and efficiency.

InfiniBand, the high performance communication architecture, is suitable for the cluster computing. The high bandwidth and low latency features fill the gap between processing power and I/O throughput. The user-level programming interface eliminates domain crossings between user-space and kernel-space. With this so-called zero-copy technique, the transmitted data is no longer need to be moved several times in memory. Many researches involved for the appearance of InfiniBand. Some studies [9, 13, 14] on DSM use different features of InfiniBand to improve performance.

2.2 Software DSM Systems

Software DSM systems can be divided into two types. One is what we follows, the page-based, and the other is object-based. The object-based DSM, for example Adsmith [10], needs explicit operations to access shared data. Although this method can only do the truly needed communication with efficiency, but in other word, the consistency of shared data is maintained by programmers. The programming model of object-based is not so similar to the one of traditional shared-memory. Opposite to

the object-based DSM systems, programming on the page-based DSM systems is much resembled to shared-memory, and saving the efforts to modify the original programs. But it also has drawbacks, like OS dependency, unnecessary communication caused by false sharing, extra computation overheads. All these factors can heavily affect the performance. However, numerous researches tried to mitigate the effects caused by above problems.

IVY [7] is the first page-based software DSM system which uses the write invalidate update protocol and implements multiple-reader - single-writer semantics. Since it follows the strictly consistency model, the effects of false sharing can lead to lots amount of communication and downgrade the parallelism and efficiency. To address this problem, the relaxed consistency model provides finer-grained data access with the price of little programming restrictions. One of the relaxed consistency model is lazy release consistency (LRC) [6], which delays the propagation of dirty pages till the ACQUIRE operation. That is, the fresh data can only be guaranteed right after the synchronization operations. To alleviate the false sharing effects, most consistency protocols support a multiple-writer scheme, which allows two or more different nodes write to the same page at same time. This scheme preserves the original copy of unmodified page for later to find out the differences, which can be collected and merged to update pages. In the traditional

LRC multiple-writer protocol, the ACQUIRE operation will collect the diffs that are distributed among other nodes. Since the diffs can exist on many nodes, the action of collecting diffs can induce longer latency. Moreover, the diffs may occupy extra memory space, since they can safely be discarded only when all nodes are synchronous.

Home-based lazy release consistency [4] protocol can make up the deficiency of the LRC multiple-writer scheme mentioned in the pervious paragraph. In the HLRC protocol, each of the shared page is assigned a home node that collects all the diffs of the page. By this centralized management, the ACQUIRE operation need not to ask many nodes to get all the diffs, but only ask the home node. Besides the communication advantages, the modifications on home node do not waste computation cycles to create twin, and avoid generating diffs afterward. If a page is written by one node in most of time, the home of that page should be assigned to the node. In this way, the communication and computation resources are used more efficiently. However, deciding the home node of the pages can heavily affect the performance [11, 12].

There are researches [5, 15] taking a totally different approach in the distributed memory environment. Instead of retrieving the data from remote nodes as the traditional "owner computes" paradigm, the executing entity is migrated to the remote

node to access data. Since most DSM systems are based on SPMD model, the migration process can be cheaper and easier. When a thread migration takes place, only the status information of the thread is needed on the target node for restoring the execution. The status information includes the instruction pointer, the stack frame, and the values in some registers. Some DSM systems [2, 17] adopt thread migration for dynamic load balancing and communication traffic minimization.

However, adopting the approach of thread migration within relaxed consistency model introduces a problem [17], that is, if a thread migrates to the remote node without any extra information, the thread will lose the pervious modified data. It can be illustrated by Figure 2.1. To conform to the rules of relaxed consistency, the migration of a thread will cause much work for maintaining the memory consistency. To alleviate the overhead, the migrations can be carried out only when the synchronization operations are executed.

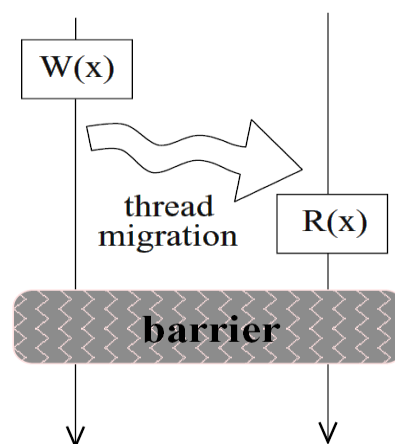


Figure 2.1: The problem of thread migration within RC model

Chapter 3

Architecture of Our System

Our DSM system, named MLDSM (Multithreaded, memory Locality concerned DSM), is consisted by three components, threading subsystem, memory management subsystem, and communication subsystem. The DSM system is implemented in C++, and the three subsystems are individually coded as three classes in MLDSM. We would introduce them in the following sections.

3.1 The Threading Subsystem

In consideration of performance effectiveness, we built a simple user-level threading system instead of the off-the-shelf kernel-level one. The system adopted the simplest scheduling algorithm, *the first-come, first-served* algorithm. By using the non-preemptive scheduling scheme, the time of unnecessary context switching can be reduced to the minimum. Thread switches can only happen at the time of waiting for a requested page or for the entry of a critical section. Since MLDSM is aimed to a dedicated computing environment, the response time is not the major concern. Adoption of the concept of multithreading in DSM systems [1, 16] can overlap the communication and computation. The overlap hides the communication latency by keeping the processor busy and makes the system more efficient.

All the threads created in MLDSM are mapped to one kernel-level thread. The threading system provides a time-shared environment. To hide the communication latency, the execution of threads can be independently divided into several intervals. Threads on a node are sharing the same view of memory. When two or more threads want to access the same page, they may send out the same requests to the remote node. This behavior causes unnecessary work for handling the surplus requests. To solve this problem, the MLDSM records the sent out requests, and all following same requests are blocked and recorded. When the remote node replies, the properly responds will be made to the requested threads according to the previous records. Both the communication and computation resources can be saved by this method.

The threading system currently only supports the x86 architecture. For efficiency and correctness of the context switching among the threads, assembly code is used to implement the switching behavior. In order to support thread migration for providing an alternative remote memory access approach, the stored information are well packed in blocks for later transfer.

3.2 The Memory Management Subsystem

The memory management subsystem takes care of the consistency of shared pages

and provides some synchronization methods. To be compatible with the interface of Pthread, the similar operations are provided, including mutual exclusive lock, conditional variable, and barrier. These operations are integrated with the HLRC protocol for ease of programming complexity. The existing parallel programs can be run on the DSM system with few modifications.

MLDSM adopts the HLRC multiple-reader - multiple-writer protocol, which is suitable for the InfiniBand. Since the home-based scheme is used, the most up-to-date pages can be fetched from one node. The RDMA operations of InfiniBand can well function on this kind of requirements. In the multiple-writer protocol, the release operation will find out the diffs between the dirty pages and the pre-created twins. The diffs should be packed before sending to the home node if traditional network interface is used. Not only extra buffer is needed, but also wasting computing cycles for data movement. With the support of InfiniBand, the noncontiguous modified data, the diffs, can be sent to the remote node without consuming the extra buffer space and the cycles of a processor.

To be simple and efficient, MLDSM views the memory address as the same one on different nodes. In other words, the addresses are reserved for unitary that can avoid dealing with the problem of address translation. With the compiler support, an extra section is inserted into the ELF file format to provide the memory region for

global variables. Although reserving the memory space can achieve efficiency of computation, but lose the efficiency of space usage. With the concern of performance and InfiniBand support, the iso-address scheme is chosen finally.

3.3 The Communication Subsystem

With the support of Mellanox VAPI library, we could have the InfiniBand more effective.

The major feature of InfiniBand to achieve efficiency is host-assisted zero-copy transfer. In order to take advantage of this feature, the communication subsystem cooperates with the other two subsystems. The most important function provided by the communication subsystem is `tell()`, which has the same functionality of `send()`. The communication subsystem doesn't provide the receive function for use by other subsystems. There is another thread to receive all incoming messages and notify the correlative subsystem according to the content of messages. Since the user-level threading system is used, a blocking call can block all other threads on the same node.

Using the software transport verbs of InfiniBand in user-level programs can offload much overhead from the OS. But it also means that the resources are no longer managed by the OS. The programs have to use the relative low-level functions to do the communication. All the transmitting operations can only address

the "registered" memory. The registration of memory pins the pages in the physical memory so that the InfiniBand can safely access the memory buffer through the virtual address. The programs using the InfiniBand should make good management of the memory buffers to store the transmitting data. In MLDSM, all the shared pages are registered at the beginning to avoid the latency of on-demand memory registration. One node can fetch any shared page on any other nodes without requests by RDMA operations if the shared pages are registered in advance. Besides the shared pages, the extra buffer for use of communication messages is also registered. The message buffer allocation interface is also provided for internal use.

Chapter 4

Memory Locality Concerned Approach

The biggest difference between the DSM system and the traditional shared memory system is that the memory data in a DSM system can not always be accessed immediately. Accessing remote data greatly delays the computing progress, and should be kept away for performance effectiveness. However, it is impossible to eliminate all communications in the environment of a DSM system, unless putting all tasks on the same node. The arrangement indeed needs no communication and increases data locality, but is totally meaningless to parallel programs. To minimize communication and avoid imbalanced load, dynamic reconfiguration of the system is needed. We take thread migration as the solution to solve this problem.

In this chapter, we propose an approach by taking the memory locality as the major concern for using computing resources efficiently. The software page-based DSM system handles the page fault event, which is raised when accessing invalid pages, to see when to fetch remote data. This process waste computing cycles, and introduce communication latency. The strategy in this approach is to reduce the times of page fault occurs by gathering temporal related pages together. If a thread would like to access remote data, which are all in the group we gathered before,

instead of several times of page fault and transfer for several pages, simply move the execution entity to remote node can be beneficial.

The thread migration can be favorable if the amounts of needed data are large enough. It depends on the characteristics of applications. The matrix multiplication and pixel averaging applications show good results [5] by adopting the approach of thread migration. The favorable situation of thread migration can be shown in Figure 4.1. This Figure describes that the approach of thread migration can reduce the times of page fault happened if a thread want to access several remote pages.

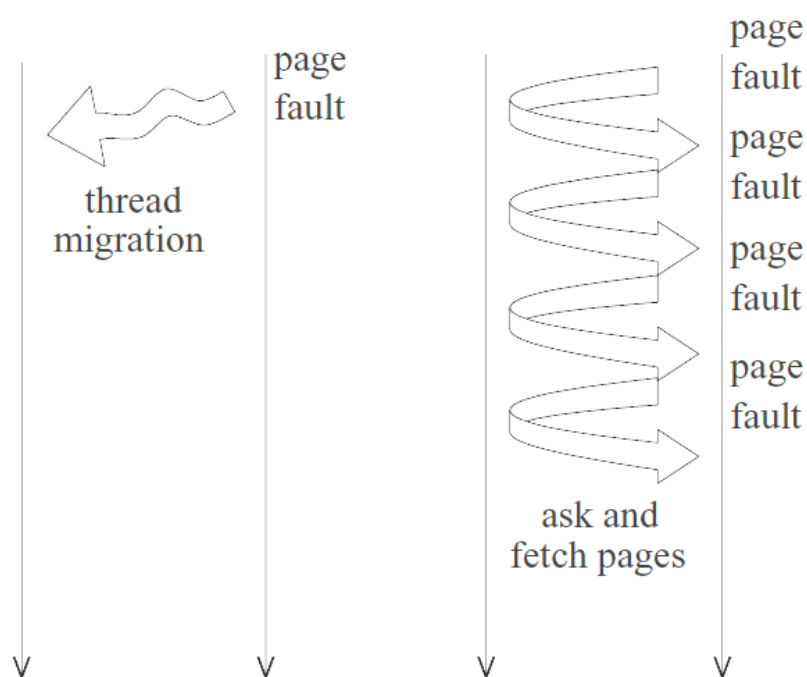


Figure 4.1: The advantage of thread migration.

There are still some situations that are not suitable for adopting the thread migration approach. It can be seen in Figure 4.2. When a thread is accessing two variables

that are located on different pages, using thread migration can result in the ping-ponging of the threads. In this case, the thread migration should be avoided.

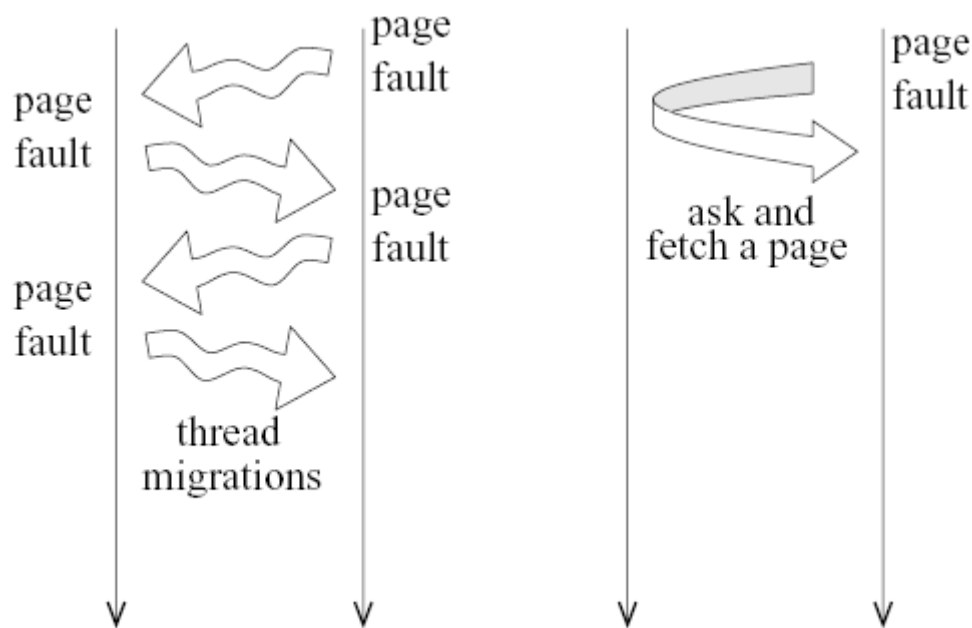


Figure 4.2: The advantage of page fetching.

However, it is hard to determine the situations shown above. The information provided by a DSM system is not sufficient to identify the two. So, we tried to limit the ping-ponging, and hope the first situation happened.

As Pete Keleher mentioned in [17], the problem to fine out the arrangement of resources that maximizes parallelism, minimizes load imbalance, and minimizes communication can be mapped to the *multi-way cut* problem, which is NP-hard. To enhance the memory locality, we face a similar problem. If we just put our minds on

the problem of memory locality, the load of all nodes can be unbalanced and induce worse performance. The following two are what we concern about for achieving better performance.

- Reduce the times of page fault happened to use the available resources efficiently.
- Keep all nodes busy to maintain the parallelism.

The memory sharing information can only be tracked by the events of page fault. In our scheme, all pages on each node are assigned a dynamics degree. Occurrence of page fault will lead to the increment of dynamics degree of the page. The dynamics degree represents the stability of a page. High degree of dynamics means the page is shared by many threads on different node. For this kind of pages, the content will be duplicated from its home node and adopt the multiple-writer scheme to maintain the parallelism. The home is also fixed, since the location of home does not matter too much. For being adaptive to the access pattern of the threads, the page with low degree of dynamics will be migrated to the accessing node.

According to the requests for remote pages and the page fault events on the home node, the affinities between the threads and nodes are also recorded. The record consists of the first three high affinity nodes for one thread. Large numbers of

consecutive requests will be recorded for high affinity. The values of affinity are used to reconfigure the arrangement of threads and home of pages.

It can be a problem if a node is the home of almost all pages. This will induce a super affinity for all other threads. In such a situation, all threads probably move themselves to the charming node and make the DSM system imbalance and inefficient. If the situation happens, we take a gentle scheme to avoid the rash. When the remote affinity of a thread is higher than local one, the thread will become a "trailblazer". The "trailblazer" will try to collect remote pages onto the node it located. If the collecting process works, the remote affinity will increase slowly. Or when the remote affinity is higher than the threshold, the thread will probably be migrated in the next barrier operations.

We briefly describe the work flow of MLDSM in the following. At the beginning, the created threads are evenly spread among all nodes using the round-robin scheme. The degree of dynamics of each page is set to -1 to support the *first-touch* home assignment. The requested node will then migrate the home to the requesting node if the dynamics degree is -1. According to the access patterns of the threads, the dynamics degree of pages shared by many threads on different nodes will increase much faster than the pages shared by fewer threads. At the same time, the number of requests for remote pages is also recorded for evaluating the affinity between

threads and nodes. Besides the migrating due to the *first-touch* scheme, the normal request of page can also migrate the home. The dynamics degree also represents the desire of the page. The higher degree of the page, the more threads need it. The requested node compares the values of dynamics degree, and assigns the home to the one with higher degree.

When the degree of dynamics of a page reaches the threshold, this page is thought to be the multiple-writer page. Migrating the home of the page with high degree can not provide better performance. Since the page is accessed by several nodes, no matter where the home is located, there is at least one node suffers from the overhead of multiple-writer protocol. By adopting thread migration, the modifications of the system configuration may possibly reduce the overhead.

A thread will be migrated to the node where its favorite pages are located if the thread is being a "trailblazer" and can not collect some of the pages. In the ideal situation, the migrated thread will bring the pages that are not so popular to the node it stays. The protocol overhead can be kept as low as it was on the original node, and the overhead due to accessing the multiple-writer page can be reduced.

Considering the balance of load, the thread migration is preferred to be an exchanging process. A migration process is always involved with two nodes.

These two nodes will try to find the threads with the highest affinity to the other node.

By the exchanging scheme, the situation of load imbalance can be reduced. In the next chapter, we will show some results of benchmark programs running in MLDSM with the proposed approach described above.

Chapter 5

Performance Evaluation

This chapter shows the performance results of the proposed DSM system.

5.1 The Experimental Environment

The testbed is a cluster with four nodes. Each node has dual Xeon 2.4GHz processors and 1GBytes of memory. All nodes run the Linux 2.6.8 operating system and are equipped with InfiniCon InfiniServ host channel adapter. All are connected via InfiniCon InfiniIO 3000, the InfiniBand switch. The InfiniBand network supports 4X (10Gbits/s) link configuration.

The costs of basic operations of MLDSM are presented in Table 5.1. The page size and the stack size of the migrating thread are about 8 kilo-byte. We can see that the cost of reading the remote page is 6 times higher than the local one. The cost can be even higher if the diff processing cost is considered. The thread migration takes about half of the time of fetching a remote page. The difference is mainly introduced by network latency. The thread migration takes only one network operation, and fetching page is a two-way operation.

Table 5.1: The costs of basic operations.

Operation	Time
handling the page fault of a local page	30 us
handling the page fault of a remote page	181 us
thread migration	93 us

The benchmark applications consist of three programs (FFT, SOR, QS) from the CVM distribution and two programs from the SPLASH-2 benchmarks suite (LU and Ocean). All these programs only need a little modification for using MLDSM. The characteristics and descriptions of all the benchmark applications are summarized in Table 5.2. The letters L, B in the "synch" column stand for lock and barrier, the synchronous operations used in the application, respectively.

Table 5.2: Characteristics of the benchmark programs

Program	Data set	Synch	Description
FFT	128×128×128	B	3-dimensional fast fourier transform
SOR	2048×2048	B	Successive over relaxation
LU	2048×2048	B	LU factorizatoin of a matrix
Ocean	514×514	L, B	simulation of ocean
QS	4194304 entries	L	quick sort

The applications only used barrier synchronous operations are mostly iterative programs with regular memory access patterns. The workload can be easily divided for this kind of programs. The divided workloads are less dependent than other programs. For the regular iterative programs, using *first touch* scheme to assign the

home node works very well. The number of requests for remote pages is small, and the affinity for remote node is also small. Migration of threads can happen with small chance for these programs. If we force the migration to happen, the programs suffer from the overheads of moving the stable pages to the new node and result in poor performance.

5.2 Performance Results

Following are Figures that show the performance results of the five programs with different number of threads and nodes. The first three programs, FFT, SOR, and LU, are with regular memory access patterns. The migration will not happen in these programs, since the affinity of remote nodes will not be higher than the local one. These programs usually perform well in parallel. In Figures 5.1, 5.2, and 5.3, the performance of Pthread on SMP with 2 threads are a little better than the MLDSM on 2 nodes. It is reasonable that the DSM system always cannot avoid the overhead introduced by internode communication.

The results of FFT, and LU show the best performance with 4 threads per node. Multithreading improves the performance by hiding the communication latency. Although InfiniBand provides the feature of low latency, the effects still can not be ignored. Considering SOR, the performance gets worse when the number of threads increases. The reason is that the access pattern of SOR is coarse-grain.

Each thread access a large continuous set of data, and only the boundaries of data is shared. Since the shared data is small, the communication introduced by fetching shared data is also small. The latency hiding can not gain benefit significantly. But multithreading introduces more overheads caused by context switching and synchronization. We can see that all these programs performs badly if the number of threads gets too high.

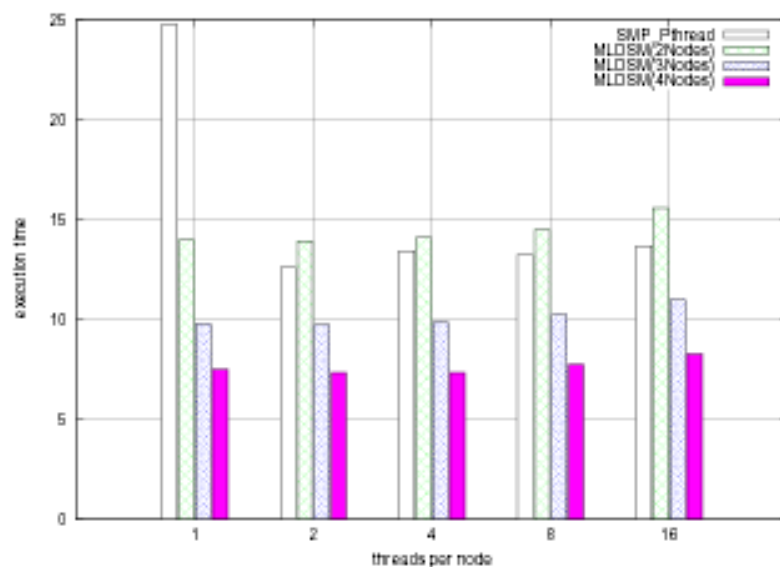


Figure 5.1 The results of FFT

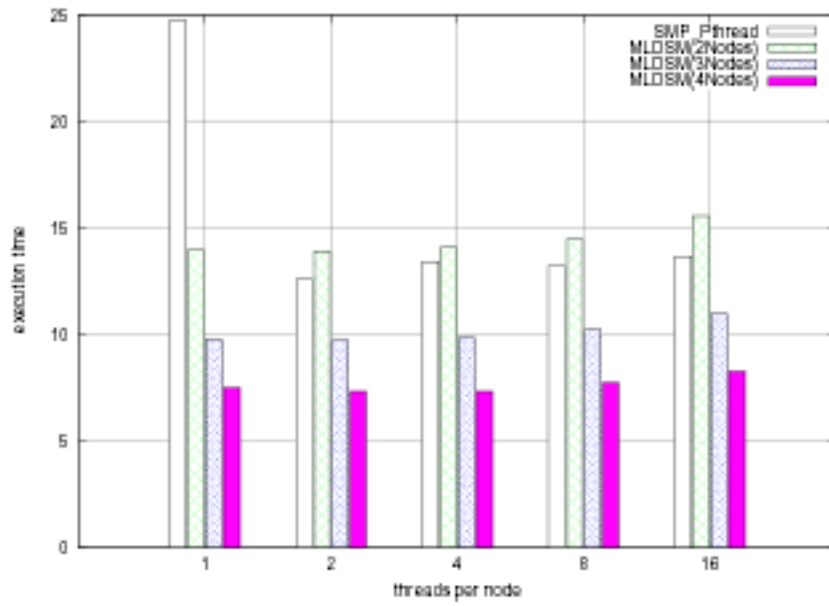


Figure 5.2: The The results of QS

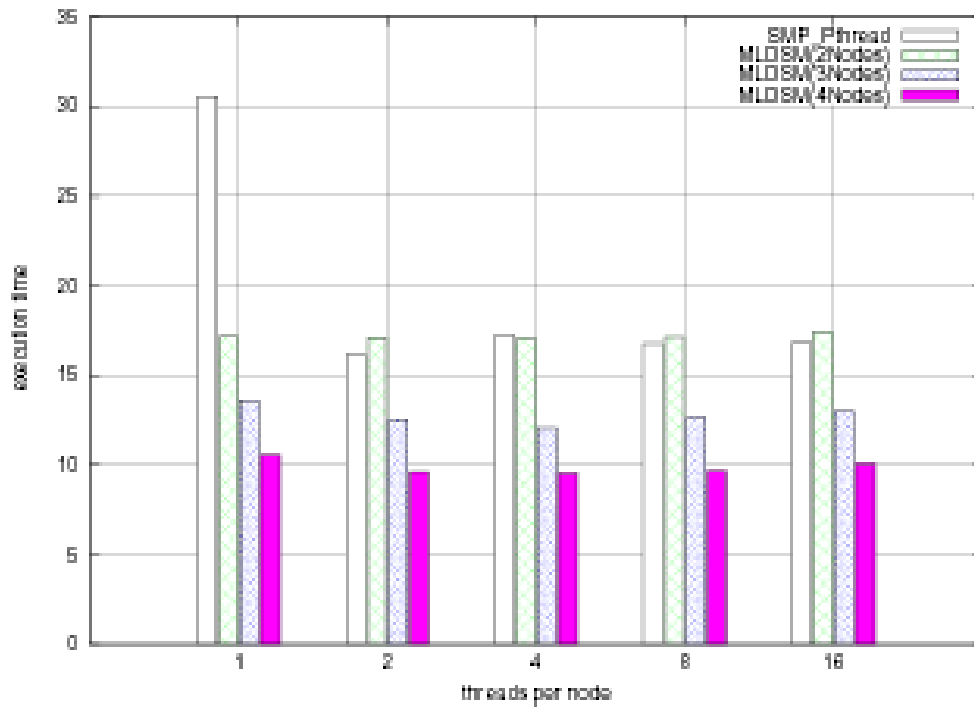


Figure 5.3: The results of LU.

Adopting the multithreading scheme, the performance of above programs is improved. However, for the programs with finer-grain and irregular access patterns, multithreading can introduce overheads. Since the threads on one node share the

same address space, the false sharing can be happened among the local threads. A thread in the critical section may invalid a page, which contains some variables that is used by other local threads. It makes all threads that are accessing the page blocked and idle till the page is fetched from remote node.

The following results show the poor performance when adopting multithreading scheme. The QS application only uses lock as the synchronous operation. Since we tried to migrate a thread within barrier operations to avoid extra overheads introduced by the consistency protocol. The thread migration can not happen in QS. Figure 5.4 shows the results with poor performance when the thread number increases. It is mainly due to the false sharing among local threads.

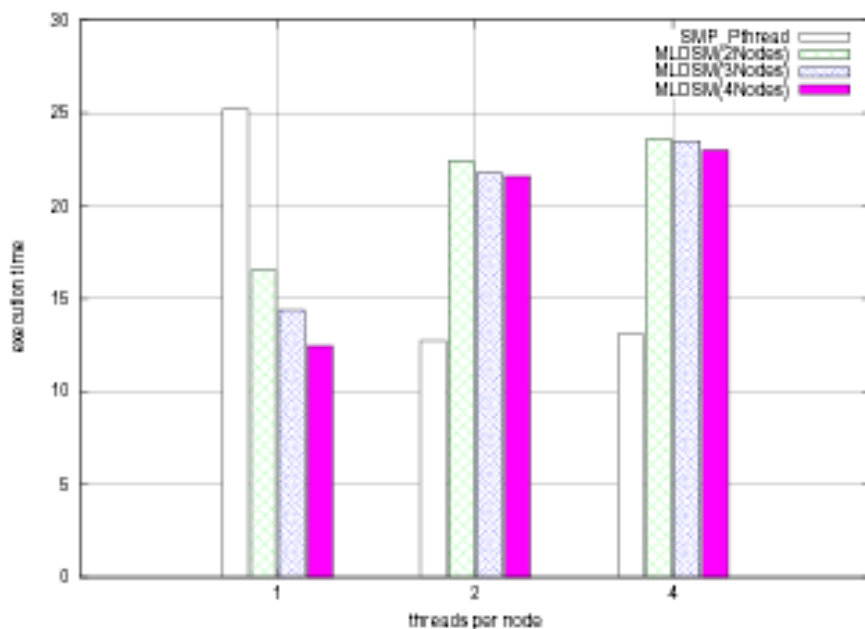


Figure 5.4: The result of QS

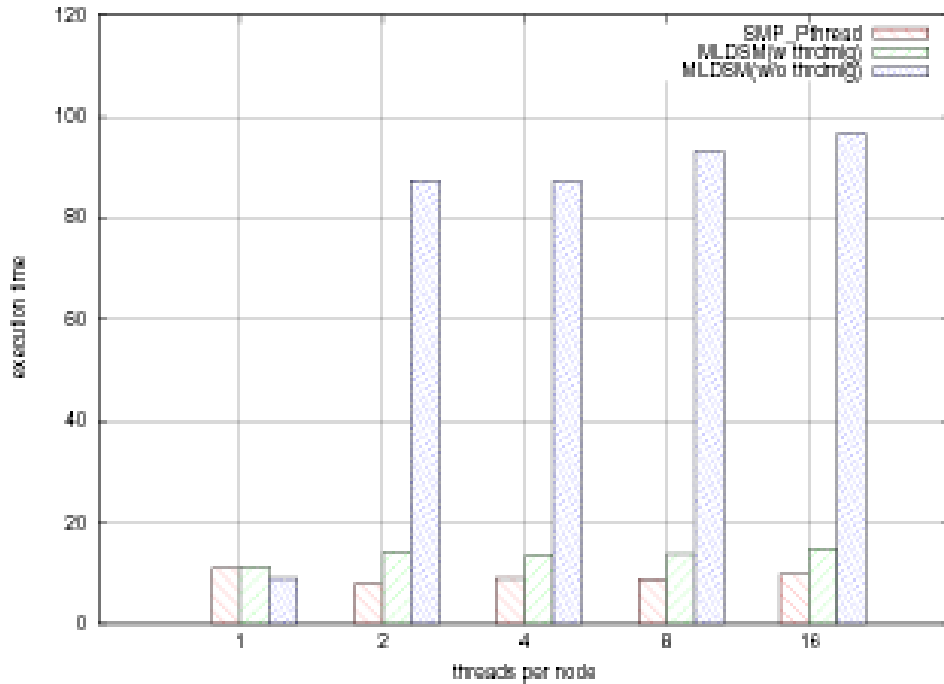


Figure 5.5: The result of OCEAN

Figure 5.5 shows the execution time of the program, OCEAN, running on two nodes using different scheme with different number of threads. The result shows much worse performance when multiple threads are fixed on the original node. The program contains large amounts of barrier operations that make even more overheads in the distributed environment. In this program, the pages are heavily shared among the threads and induce great amounts of communication traffics. The requests for remote pages are so many and make the remote affinity increase rapidly. Collecting the pages by being the "trailblazer" can not cure the traffics and the remote affinity still increases. Detecting the high affinity to remote node within the barrier operation makes the thread migrating to the remote node to alleviate the heavy traffic of communication.

The results show that the thread migration can be helpful to detect the expansive communication overheads and avoid suffering in such situation. Also our scheme can distinguish the programs with coarse-grain or fine-grain access patterns, and adopt the proper method to reduce the overheads.

Chapter 6

Conclusions and Future Work

In this thesis, we introduced a page-based DSM system named MLDSM that can be adapted to the behavior of programs by adopting migration of pages and threads. With the support of InfiniBand, overheads can be reduced due to the capacity of zero-copy. The MLDSM is designed with the interface of the POSIX threading standard. The general shared-memory programs can be ported to the MLDSM without amounts of modification.

Our implementation still has large improve space. The ability of threading system in DLDSM should be extended in the future. The user-level threads managed by threading system do not map to two or more physical processor. In other words, the threading system does not support the symmetry multiprocessor architecture. Exhausting the processing power on each node can make the multithreaded programs reach even higher performance. The scalability should be further evaluated with more nodes.

References

- [1] G. Antoniu, L. Bougé, DSM-PM2: A portable implementation platform for multithreaded DSM consistency protocols. In *Proceedings of the 6th International Workshop on High-Level Parallel Programming Models and Supportive Environments (HIPS '01)*, pp. 55-70, 2001.
- [2] G. Antoniu, C. Perez, Using Preemptive Thread Migration to Load-balance Data-parallel Applications. In *Proceedings of the 5th International European Conference on Parallel Processing (Euro-Par '99)*, pp. 117-124, 1999.
- [3] K. Gharachorloo, D. Lenoski, J. Laudon, P. Gibbons, A. Gupta, and J. Hennessy, Memory Consistency and Event Ordering in Scalable Shared-Memory Multiprocessors. In *Proceedings of the 17th International Symposium on Computer Architecture (ISCA '90)*, pp. 15-26, May 1990.
- [4] L. Iftode, Home-based Shared Virtual Memory. Ph. D. thesis, Princeton University, Aug 1998.
- [5] S. Jenks, J. L. Gaudiot, Nomadic Threads: A Migrating Multithreaded Approach to Remote Memory Accesses in Multiprocessors. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT '96)*, pp. 2-11, October 1996.
- [6] P. J. Keleher, A. L. Cox, and W. Zwanepoel, Lazy Release Consistency for Software Distributed Shared Memory. In *Proceedings of the 19th International Symposium of Computer Architecture (ISCA '92)*, pp. 13-21, May 1992.
- [7] K. Li, IVY: A Shared Virtual Memory System for Parallel Computing. In *Proceedings of the 1988 International Conference on Parallel Processing*, vol. 2, pp. 94-101, August 1988.
- [8] Chang-Yi Lin, An Adaptive Migratory Home Protocol for Software DSM Systems. Master thesis, Tsing-Hua University, Jun 2004.
- [9] L. Liss, Y. Birk, A. Schuster, Efficient Exploitation of Kernel Access to InfiniBand: a Software DSM Example. In *Proceedings of the 11th Symposium on High Performance Interconnects (HOTI '03)*, pp. 130-135, August 2003.
- [10] W. Y. Liang, C. T. King, and F. Lai, Adsmith: An Object-based Distributed

- Shared Memory for Network of Workstations. In *IEICE Transaction on Information and Systems*, vol. E80-D, no. 9, pp. 899-908, Sep 1997.
- [11] Y. T. Liu, T. Y. Liang, C. T. Huang, C. K. Shieh, Memory Resource Considerations in the Load Balancing of Software DSM Systems. In *Proceedings of the 2003 International Conference on Parallel Processing Workshops (ICPPW '03)*, pp. 71-78, 2003.
- [12] D. S. Nikolopoulos, C. D. Polychronopoulos, Scheduler-Activated Dynamic Page Migration for Multiprogrammed DSM Multiprocessors. In *Journal of Parallel and Distributed Computing*, vol. 62, pp. 1069-1103, June 2002.
- [13] R. Noronha, D. K. Panda, Designing High Performance DSM Systems using InfiniBand Features. In *Proceedings of the 4th International Symposium on Cluster Computing and the Grid (CCGrid '04)*, pp. 467-474, April 2004.
- [14] C. Osendorfer, J. Tao, C. Trinitis, M. Mairandres, ViSML: Software Distributed Shared Memory for InfiniBand Clusters. In *Proceedings of the 3rd International Symposium on Network Computing and Applications (NCA '04)*, pp. 185-191, August 2004.
- [15] A. Rogers, M. C. Carlisle, J. H. Reppy, L. J. Hendren, Supporting Dynamic Data Structures on Distributed Memory Machines. In *ACM Transactions on Programming Languages and Systems*, vol. 17, no. 2, pp. 233-263, 1995.
- [16] K. Thitikamol, P. J. Keleher, Multi-threading and Remote Latency in Software DSMs. In *Proceedings of the 17th International Conference on Distributed Computing System (ICDCS '97)*, pp. 296-304, 1997.
- [17] K. Thitikamol, P. J. Keleher, Thread Migration and Communication Minimization in DSM Systems. In *Proceedings of the IEEE, Special Issue on Distributed Shared Memory*, pp. 487-497, 1999.
- [18] L. Whately, R. Pinto, R. Bianchini, C. Amorim, Adaptive Techniques for Home-Based Software DSMs. In *Proceedings of the 13th Symposium on Computer Architecture and High-Performance Computing*, September 2001.
- [19] Y. Zhou, L. Iftode, and K. Li, Performance Evaluation of Two Home-Based Lazy Release Consistency Protocols for Shared Virtual Memory Systems. In *Proceeding of the 2nd Symposium on Operating System Design and Implementation (OSDI '96)*, pp. 75-88, October 1996.